

From Records and Abstract Data Types to Objects

(in Java) Part 1

Paul Curzon

Abstract data types are the basis of objects and so of object-oriented language. The first (of many) advantages that OOP languages give is to provide compiler support for the creation of abstract data types. They allow the methods of an abstract data type (ADT) to become part of the record definition. To start with you can think of an object as being a record + methods. Eventually you will see it is a lot more than that.

You cannot just move the methods from one class to the other though, as you need new syntax - the syntax of objects, and in particular a new way of writing calls. There is also a whole new terminology - new words to refer to the same things. Part of the reason for this is that the changes in the way you write the program either are about letting the compiler do some things for you, or giving the compiler extra information about your intentions that it can then check.

A key thing to realise is that underneath all these surface changes the same things are happening in the program, you are just using a different way of writing the program to tell the computer instructions that do the same things. Later you will see how it also opens up new opportunities.

Objects and Instance Variables

The first change in terminology is that the records are now called **objects** and the fields of the record are now called **instance variables**. Essentially the same thing is going on though.

Whenever you create a **new** object you create a new set of instance variables (the things we previously called field names) that hold the state of the object.

Defining Constructors

If you have been writing 'create' methods as part of each ADT that both create the new object (allocate space) and initialise values for the fields, then you have been using something equivalent to what we call '**constructors**' in OOP terminology.

A constructor is just a special method. It is the one that is called automatically when you use **new** in Java. The name of the constructor is also always the name of the type of the object. So if you have a student ADT creating a type called Student, then its constructor is also called Student.

If converting a program in the procedural style we have been following, you can take your create method (createStudent) and move it from the main class to the Student class. You have to make a series of changes to the syntax though as follows:

1. Remove the line that calls new in the constructor. Because it is the constructor that compiler knows to do that automatically as the first step.
2. You now just use the instance variable names to refer to them within the body of the constructor. From the fact that they are in the constructor, we know that they refer to the particular instance variables of the object being created. Therefore, in the constructor, rather than writing `s.name` and `s.id` you just write `name` and `id`.
3. Rename the method Student so it is exactly the same as the name of the class and so the type being created. This is how the compiler knows you are defining a constructor and not something else.
4. Get rid of the keyword **static** from the front of the header line.
5. Also get rid of the **return** statement (it is now done automatically for you by the compiler - that is what a constructor is for to give back the new object)

So for example a method to create a record that we might have written before in the main class as:

```

public static Student createStudent (String studentname, int studentid)
{
    Student s = new Student();
    s.id = studentid;
    s.name = studentname;
    s.courseworkmark = 0;
    s.exammark = 0;
    return s;
}

```

when written as a constructor might become:

```

public Student (String studentname, int studentid)
{
    id = studentid;
    name = studentname;
    courseworkmark = 0;
    exammark = 0;
}

```

Calling Constructors

Now for the call you just use the normal syntax for new. The keyword new followed by the constructor name (eg `new Student ( )`) actually calls the constructor method itself automatically. Any arguments that the createMethod needed are put in the brackets as arguments just as they were with the create method.

So you now write

```
Student s = new Student("Paul", "673462");
```

for example, instead of the old version

```
Student s = createStudent("Paul", "673462");
```

Exactly the same thing is happening we are just using a different syntax to say the same thing. Now the compiler knows what you are trying to do it can help make sure you only do it when it makes sense.

A Full Program

Putting all this together we get programs such as the one below. We have bolded the parts that have changed from an equivalent program written in the procedural style. In Part 2 we will look at going a step further.

```

/* *****
AUTHOR Paul Curzon
This program demonstrates how in an object-oriented program constructors
can replace specific methods to create a new object
*/
import java.util.Scanner;

class studentconstructor
{
    // A simple test method setting a student record then printing it out

    public static void main(String []p)
    {
        Student s0 = new Student("Martin Luther King", 3921072);
        System.out.println(StudentToString(s0));

        Student s1 = new Student("Barack Obama", 4509930);
        int mark1 = 49;
        int mark2 = 50;
        setMarks(s1, mark1, mark2);
        System.out.println(studentToString(s1));

        return;
    } // END main

    /* *****

    ** Methods defining a Student ADT
    ** Theses are the only ways we should access a Student
    ** Get a student record as a formatted string
    ** Get a student name
    ** Get a student ID
    ** Get a student's combined mark
    ** Create a student - now part of the object class definition
    ** Set the marks for a student
    **/

    // Convert a student record to a printable string
    //
    public static String studentToString (Student s)
    {
        String message = getId(s) + "\t" + getName(s) + "\t" + getMark(s);
        return message;
    } // END studentToString

    // Getter methods for Student record type
    // Return the name from a student record
    //
    public static String getName (Student s)
    {
        return s.name;
    } // END getName

    // Return the id from a student record
    public static int getId (Student s)
    {
        return s.id;
    } // END getId

```

```

// Return the total mark from a student record
// This is the only mark accessible
public static int getMark (Student s)
{
    return (s.exammark + s.courseworkmark); // WE CHANGED THIS
} // END getMark

// Set the mark of a student by giving exam and coursework mark /50
returning the updated record
//
public static void setMarks (Student s, int smark1, int smark2)
{
    s.courseworkmark = smark1;
    s.exammark = smark2;

    return;
} // END setMarks

} //END class studentrecord

/* ***** */
/* ***** */

/* *****
    Create a new type (an object) called Student that records student data
    A student has a name and an ID and two marks.
*/

class Student
{
    String name; // The Students full name
    int id;      // Their unique ID number
    int courseworkmark; // their coursework mark /50
    int exammark;    // their exam mark /50

    // CONSTRUCTOR
    // To create a student BOTH a name and ID must be set at the same time
    // The marks are set to 0 to mean no mark set.
    //
    public Student (String studentname, int studentid)
    {
        id = studentid;
        name = studentname;
        courseworkmark = 0;
        exammark = 0;
    } // END Student
} // END class Student

```